BIKEPEDIA

Bicycle Education Web Application

Project Documentation and Development Plan

Project Area	Current Plan
Project Type	Educational / information web application
Main Goal	Help users understand bicycle types, components, materials, brands and selected models
Version 1	2D responsive web application with browsing, search, filtering and bicycle comparison
Future Extension	Interactive 3D bicycle explorer and animated mechanical systems

1. Project Overview

BikePedia is a web-based educational application for people who want to understand bicycles. The project brings information about bicycle types, components, materials, brands and selected models into one organised website.

1.1 Problem

Bicycle information is spread across manufacturer websites, shops, forums, videos and technical articles. This can make it difficult for a beginner to understand what a bicycle part is, what it does, how it works and how different bicycles compare.

1.2 Proposed Solution

I will build a responsive web application where users can browse bicycle types, learn bicycle anatomy and components, read about common materials, explore selected brands and models, search and filter the stored information, and compare bicycles side-by-side.

1.3 Version 1 Scope

Included in Version 1	Not in Version 1
Bicycle types and type detail pages	A catalogue of every bicycle in the world
Bicycle anatomy and components	E-commerce, payments or delivery
Component purpose, operation and materials	User or administrator accounts
Material information	Full interactive 3D bicycle
Selected brands and representative models	Real-time retailer inventory
Search and filtering	Teaching users how to ride
Bicycle comparison	

1.4 Scope Decision

I am keeping Version 1 focused so I can complete the main features properly. The 3D explorer is a future extension and will not block the first working version.

2. Requirements

2.1 Functional Requirements

ID	Requirement
FR-01	The system shall allow users to browse bicycle types.
FR-02	The system shall allow users to view bicycle component information, including purpose, operation and common materials.
FR-03	The system shall allow users to browse selected bicycle brands and models.
FR-04	The system shall allow users to search bicycle information stored in the application.

FR-05	The system shall allow users to filter bicycles by available properties such as type, brand and material.
FR-06	The system shall allow users to select bicycles and compare their stored information.

2.2 Non-Functional Requirements

ID	Quality	Requirement
NFR-01	Usability	The interface should be understandable to users with little bicycle knowledge.
NFR-02	Performance	Search, filtering and normal navigation should respond quickly with the Version 1 dataset.
NFR-03	Responsive Design	The application should remain usable on desktop, tablet and mobile screens.
NFR-04	Maintainability	Code and data should be organised so new bicycle information can be added without major rewriting.
NFR-05	Reliability	Empty searches, missing data and invalid selections should be handled without crashing the application.

2.3 Actor and Main Use Cases

The primary actor is the User. Version 1 does not require an administrator, seller, payment service or delivery service.

Use Case	User Goal
Browse Bicycle Types	Understand the main bicycle categories and their uses.
View Components	Understand a bicycle part, its purpose, operation and common materials.
Browse Bicycles	Explore selected brands and representative models.
Search	Find stored bicycle information.
Filter	Narrow bicycle results by stored properties.
Compare Bicycles	View selected bicycles side-by-side.

2.4 Detailed Use Case - Compare Bicycles

Field	Description

Objective	Allow a user to compare stored information about selected bicycles.
Precondition	Bicycle data is available in the system.
Main Flow	1. User browses or searches bicycles. 2. User selects one bicycle. 3. User selects another bicycle. 4. User requests comparison. 5. The system retrieves the stored information. 6. The system displays the bicycles side-by-side.
Alternative Flow	If too few bicycles are selected, the system asks the user to select another bicycle.
Post-condition	The selected bicycles and their comparable information are displayed.

3. System Design

Before coding, I planned the main pages and how users will move through the application. This gives each part of the website a clear purpose.

3.1 Sitemap

Home
├── Bicycle Types
│ └── Bicycle Type Detail
├── Components & Anatomy
│ └── Component Detail
├── Bicycles
│ ├── Bicycle Detail
│ ├── Search / Filter
│ └── Compare Bicycles
├── Materials
│ └── Material Detail
└── Brands
└── Brand Detail

3.2 Main Pages

Page	Purpose
Home	Introduce BikePedia and provide navigation to the main areas.
Bicycle Types	Display the main bicycle categories.
Bicycle Type Detail	Explain a type's use, characteristics, advantages and disadvantages.
Components & Anatomy	Organise bicycle parts into understandable systems.
Component Detail	Explain what a component is, what it does, how it works and common materials.
Bicycles	Browse the selected model dataset and use

	search/filter controls.
Bicycle Detail	Display a model's brand, type, frame material and stored specifications.
Compare Bicycles	Display selected bicycles side-by-side.
Materials / Material Detail	Explain common bicycle materials and where they are used.
Brands / Brand Detail	Introduce selected manufacturers and the models represented in the dataset.

3.3 Design Principle

The pages will display structured information from the application data instead of hard-coding separate information into every page. This keeps the content consistent and makes it easier to add more data later.

4. Database Design

I designed the database around the information required by the Version 1 pages and features. The design separates brands, bicycle types, materials, bicycles and components so the same information does not need to be repeated.

4.1 Tables

Table	Main Fields	Purpose
brands	brand_id, name, country, description	Stores bicycle manufacturer information.
bicycle_types	type_id, name, description, best_use, advantages, disadvantages	Stores educational information about bicycle categories.
materials	material_id, name, description, advantages, disadvantages	Stores reusable material information.
bicycles	bicycle_id, brand_id, type_id, frame_material_id, name, year, weight_kg, wheel_size, brake_system, drivetrain, gears, description, image_url	Stores the selected bicycle models and comparable specifications.
component_systems	system_id, name, description	Groups components into systems.
components	component_id, system_id, name, description, purpose, operation, image_url	Stores bicycle anatomy information.
component_materials	component_id, material_id	Links components and materials.

4.2 Relationships

Relationship	Type

Brand -> Bicycle	One-to-Many
Bicycle Type -> Bicycle	One-to-Many
Material -> Bicycle	One-to-Many
Component System -> Component	One-to-Many
Component <-> Material	Many-to-Many through component_materials

The complete field descriptions, key definitions and relationship notes are kept in the separate database-design.md file. The ER diagram is stored as ER-diagram.png.

5. UI and Wireframes

I created wireframes before implementation to plan the structure of the main pages. The wireframes show the intended layout and content areas; the final styling will be completed during implementation.

The project wireframe set contains 12 screens:

- 01 - Home
- 02 - Bicycle Types
- 03 - Bicycle Type Detail
- 04 - Components & Anatomy
- 05 - Component Detail
- 06 - Bicycles
- 07 - Bicycle Detail
- 08 - Compare Bicycles
- 09 - Materials
- 10 - Material Detail
- 11 - Brands
- 12 - Brand Detail

The exported wireframe images are stored in the BikePedia-Wireframes folder.

6. Application Architecture

The application will use a layered structure so the user interface, application logic and database access have separate responsibilities.

6.1 Planned Request Flow

Browser
  ↓ HTTP request
Controller
  ↓
Service
  ↓

Repository
↓
PostgreSQL Database

Data returns through the application
↓
Controller
↓
Thymeleaf template
↓
HTML page displayed in the browser

6.2 Main Responsibilities

Part	Responsibility
Browser / UI	Displays pages, navigation, search/filter controls and comparison results.
Controller	Receives web requests and chooses what application action is needed.
Service	Contains application/business logic.
Repository	Reads and writes application data.
Entity / Model	Represents application data such as Bicycle, Brand, Material and Component.
Thymeleaf Template	Builds the HTML page using data supplied by the application.
PostgreSQL	Stores the persistent application data.

7. Technology Selection

Area	Technology	Use in BikePedia
Language	Java	Main programming language.
Backend	Spring Boot	Builds the web application and handles requests.
Frontend	HTML, CSS, JavaScript and Thymeleaf	Creates the user interface and dynamic pages.
Database	PostgreSQL	Stores the relational bicycle data.
Database Access	Spring Data JPA	Connects Java entities and repositories to the database.
Build Tool	Maven	Manages dependencies and builds the project.
Version Control	Git and GitHub	Tracks project changes and stores the repository.
Testing	JUnit and Spring Boot Test	Supports automated application testing.

7.1 Future 3D Work

The interactive 3D bicycle explorer is not part of Version 1. I will first complete the 2D application and its core features.

8. Implementation Plan

I will build the application in small stages so each part can be run and checked before I add the next feature.

Order	Milestone
1	Create the Git repository and Spring Boot project and run the first application.
2	Understand the generated project folders, main class and Maven configuration.
3	Create the first controller and Home page.
4	Create the main Java entity/model classes.
5	Configure PostgreSQL and create repositories.
6	Add a small verified development dataset.
7	Build Bicycle Types and Bicycle Type Detail pages.
8	Build Components & Anatomy and Component Detail pages.
9	Build Brands, Bicycles and detail pages.
10	Implement search and filters.
11	Implement bicycle comparison.
12	Add responsive styling and accessibility improvements.
13	Handle empty results, missing data and invalid selections.
14	Refactor repeated code and improve project structure.
15	Run automated and manual tests and fix defects.
16	Prepare the production configuration and deploy the application.

9. Testing Plan

Testing will be carried out during development and again before deployment.

Test Area	What I Will Check
Unit Testing	Small Java logic works correctly where unit tests are useful.
Repository / Data Testing	Stored bicycles can be retrieved and filtered correctly.

Controller / Application Testing	Requests return the expected page, status and data.
Functional Testing	Browsing, search, filtering and comparison satisfy FR-01 to FR-06.
Validation / Negative Testing	Empty searches, invalid IDs and insufficient comparison selections are handled.
Responsive Testing	The main pages remain usable on desktop, tablet and mobile.
Usability Testing	A user can find bicycle information without needing instructions.
Regression Testing	Important existing features still work after changes and fixes.

9.1 Requirement Checks

Requirement	Acceptance Check
FR-01	A user can browse Bicycle Types and open a type detail page.
FR-02	A user can open a component and view its purpose, operation and materials.
FR-03	A user can browse the selected brands and bicycle models.
FR-04	Search returns matching stored information and shows a clear empty state when there is no match.
FR-05	Type, brand and material filters reduce the bicycle results correctly.
FR-06	Selected bicycles can be displayed side-by-side and invalid selections are handled.

10. Deployment Plan

After the application is complete and tested, I will prepare it to run outside my development computer.

Step	Action
1	Create a production build with Maven and run the tests.
2	Keep database credentials and other secrets outside the GitHub repository.
3	Create or connect a production PostgreSQL database.
4	Deploy the Spring Boot application to a compatible hosting service.
5	Connect the deployed application to the production database.

6	Check the main pages and features on the deployed application.
7	Add the live application link and screenshots to the project README.

11. Future Extension - Interactive 3D Bicycle Explorer

After Version 1, I may add an interactive 3D bicycle. A user could rotate and zoom the bicycle, select a component, and see information about that part. Moving systems such as the crank, chain and cassette could also be animated to show how the drivetrain works.

This is a future feature and is not required for the first version.

12. Version 1 Completion Checklist

- All six functional requirements work.
- The main pages are usable on desktop and mobile.
- The database and application data are organised and extendable.
- Search, filtering and comparison handle normal and empty/error cases.
- Important functionality has been tested.
- The GitHub repository contains meaningful commits and a clear README.
- The application has been deployed and the main features work in the deployed version.

13. Current Project Status

Requirements, system design, database design, ER diagram and the Version 1 wireframes have been prepared. The next design task is to finalise the application architecture before starting implementation.